

IO

2023-2024

Алгоритм ба Өгөгдлийн БҮТЭЦ - ЛАБ

io × UFE



```
function swap(arr, xp, yp)
{
    let temp = arr[xp];
    arr[xp] = arr[yp];
    arr[yp] = temp;
}
```

$O(1)$
 $O(1)$
 $O(1)$

```
function selectionSort(arr, n)
{
```

$O(1)$

```
    let i, j, min_idx;
```

```
    // One by one move boundary of unsorted subarray
```

```
    for (i = 0; i < n-1; i++)
    {
```

$O(N-1)$

```
        // Find the minimum element in unsorted array
```

```
        min_idx = i;
```

```
        for (j = i + 1; j < n; j++)
```

$O(N-2)$

```
            if (arr[j] < arr[min_idx]) {
```

```
                min_idx = j;
```

```
            }
```

```
        // Swap the found minimum element with the first element
```

```
        swap(arr, min_idx, i);
```

$O(3 * (N-2)) \rightarrow O_{\text{swap}}(N)$

```
    }
```

ИИТ: $O(N^2)$

```
}
```

```
function swap(arr, xp, yp)
{
    let temp = arr[xp];
    arr[xp] = arr[yp];
    arr[yp] = temp;
}

function selectionSort(arr, n)
{
    let i, j, min_idx;

    // One by one move boundary of unsorted subarray
    for (i = 0; i < n-1; i++)
    {
        // Find the minimum element in unsorted array
        min_idx = i;
        for (j = i + 1; j < n; j++)
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        // Swap the found minimum element with the first element
        swap(arr, min_idx, i);
    }
}
```

```
let arr = [64, 25, 12, 22, 11];

console.time();
selectionSort(arr);
console.timeEnd();
printArr(arr);
```

```
function bubbleSort(arr)
{
  let i, j, temp;
  let swapped = false;
  for (i = 0; i < arr.length - 1; i++)
  {
    swapped = false;
    for (j = 0; j < arr.length - i - 1; j++)
    {
      if (arr[j] > arr[j + 1])
      {
        // Swap arr[j] and arr[j+1]
        swap(arr, j, j + 1);
        swapped = true;
      }
    }
    // IF no two elements were
    // swapped by inner loop, then break
    if (swapped === false) {
      break;
    }
  }
}
```

$O(1)$

$O(N-1)$

$O(N-1)$

$O(N-2)+O(N-3)+...+O(1) = O(N^2)$

$O(N^2)$

$O(N^2) \rightarrow O_{\text{swap}}(N^2)$

$O(N^2)$

$O(N-1)$

$O(1)$

ИИТ: $O(N^2)$

```
function bubbleSort(arr)
{
  let i, j, temp;
  let swapped = false;
  for (i = 0; i < arr.length - 1; i++)
  {
    swapped = false;
    for (j = 0; j < arr.length - i - 1; j++)
    {
      if (arr[j] > arr[j + 1])
      {
        // Swap arr[j] and arr[j+1]
        swap(arr, j, j + 1);
        swapped = true;
      }
    }
    // IF no two elements were
    // swapped by inner loop, then break
    if (swapped === false) {
      break;
    }
  }
}
```

```
let arr = [64, 25, 12, 22, 11];

console.time();
bubbleSort(arr);
console.timeEnd();
printArr(arr);
```

```
function insertionSort(arr)
{
    let i, key, j;
    for (i = 1; i < arr.length; i++)
    {
        key = arr[i];
        j = i - 1;

        /* Move elements of arr[0..i-1], that are
        greater than key, to one position ahead
        of their current position */
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

$O(1)$
 $O(N)$

$O(N-1)$
 $O(N-1)$

$O(N-2)+O(N-3)+O(N-4)+...+O(1) \rightarrow O((N-1)*(N-2)/2)$

$O(N^2) \leftarrow \text{SWAP}$
 $O(N^2)$
 $O(N-1) \leftarrow \text{SWAP}$

ИИТ: $O(N^2)$

```
function insertionSort(arr)
{
  let i, key, j;
  for (i = 1; i < arr.length; i++)
  {
    key = arr[i];
    j = i - 1;

    /* Move elements of arr[0..i-1], that are
    greater than key, to one position ahead
    of their current position */
    while (j >= 0 && arr[j] > key)
    {
      arr[j + 1] = arr[j];
      j = j - 1;
    }
    arr[j + 1] = key;
  }
}
```

```
let arr = [64, 25, 12, 22, 11];

console.time();
insertionSort(arr);
console.timeEnd();
printArr(arr);
```

```
function merge(arr, l, m, r)
{
    var n1 = m - l + 1;
    var n2 = r - m;
    // Create temp arrays
    var L = new Array(n1);
    var R = new Array(n2);
    // Copy data to temp arrays L[] and R[]
    for (var i = 0; i < n1; i++) {
        L[i] = arr[l + i];
    }
    for (var j = 0; j < n2; j++) {
        R[j] = arr[m + 1 + j];
    }
    // Merge the temp arrays back into arr[l..r]
    // Initial index of first subarray
    var i = 0;
    // Initial index of second subarray
    var j = 0;
    // Initial index of merged subarray
    var k = l;
```

```
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    // Copy the remaining elements of
    // L[], if there are any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    // Copy the remaining elements of
    // R[], if there are any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}
```

```
function mergeSort(arr, l, r){
    if(l>=r){
        return;
    }
    var m =l+ parseInt((r-l)/2);
    mergeSort(arr,l,m);
    mergeSort(arr,m+1,r);
    merge(arr,l,m,r);
}
let arr = [64, 25, 12, 22, 11];

console.time();
mergeSort(arr, 0, arr.length - 1);
console.timeEnd();
printArr(arr);
```

```
function mergeSort(arr, l, r){
  if(l>=r){
    return;
  }
  var m = l+ parseInt((r-l)/2);
  mergeSort(arr, l, m);
  mergeSort(arr, m+1, r);
  merge(arr, l, m, r);
}
let arr = [64, 25, 12, 22, 11];

console.time();
mergeSort(arr, 0, arr.length - 1);
console.timeEnd();
printArr(arr);
```

$O(N \log N)$

```
function partition(arr, low, high) {
  // Choosing the pivot
  let pivot = arr[high];

  // Index of smaller element and indicates
  // the right position of pivot found so far
  let i = low - 1;

  for (let j = low; j <= high - 1; j++) {
    // If current element is smaller than the pivot
    if (arr[j] < pivot) {
      // Increment index of smaller element
      i++;
      [arr[i], arr[j]] = [arr[j], arr[i]];
      // Swap elements
    }
  }
  // Swap pivot to its correct position
  [arr[i + 1], arr[high]] = [arr[high], arr[i + 1]];

  return i + 1; // Return the partition index
}
```

```
function quickSort(arr, low, high) {
  if (low < high) {
    // pi is the partitioning index,
    // arr[pi] is now at the right place
    let pi = partition(arr, low, high);

    // Separately sort elements before
    // partition and after partition
    quickSort(arr, low, pi - 1);
    quickSort(arr, pi + 1, high);
  }
}

let arr = [64, 25, 12, 22, 11];

console.time();
quickSort(arr, 0, arr.length - 1);
console.timeEnd();
printArr(arr);
```

WORST - $O(N^2)$

AVERAGE - $O(N \log N)$

**Коктейль сэгсрэх эрэмбэлэлтийн
(Cocktail shaker sort) програмыг
бичиж Хөвөх аргын
эрэмбэлэлттэй ажилласан
хугацааг харьцуул**